FIT5225 Assignment 2 Team Report

FIT5225 2026S1 Group 92

Yuanhe Wu 36302414
Guanghui Liu 34217398
Junzhe Zhang 34682457
Zuyao Wang 34590706

Table of Contents

I. Multi-Cloud Architecture Diagram **3**
II. Roles of Team Members **4**
III. User Guide **5**
1. Accessing the Application 6
2. Uploading Media 6
3. Searching by Species or Tags 7
4. Media Results 8
5. Searching by Uploaded Query File 9
6. Searching by Thumbnail URL 9
7. Tag Management and Deletion 10
8. Notifications and Troubleshooting 11

IV. Link to Source Code (GitLab) **13**
AI Acknowledgment **13**
References **13**

I. Multi-Cloud Architecture Diagram

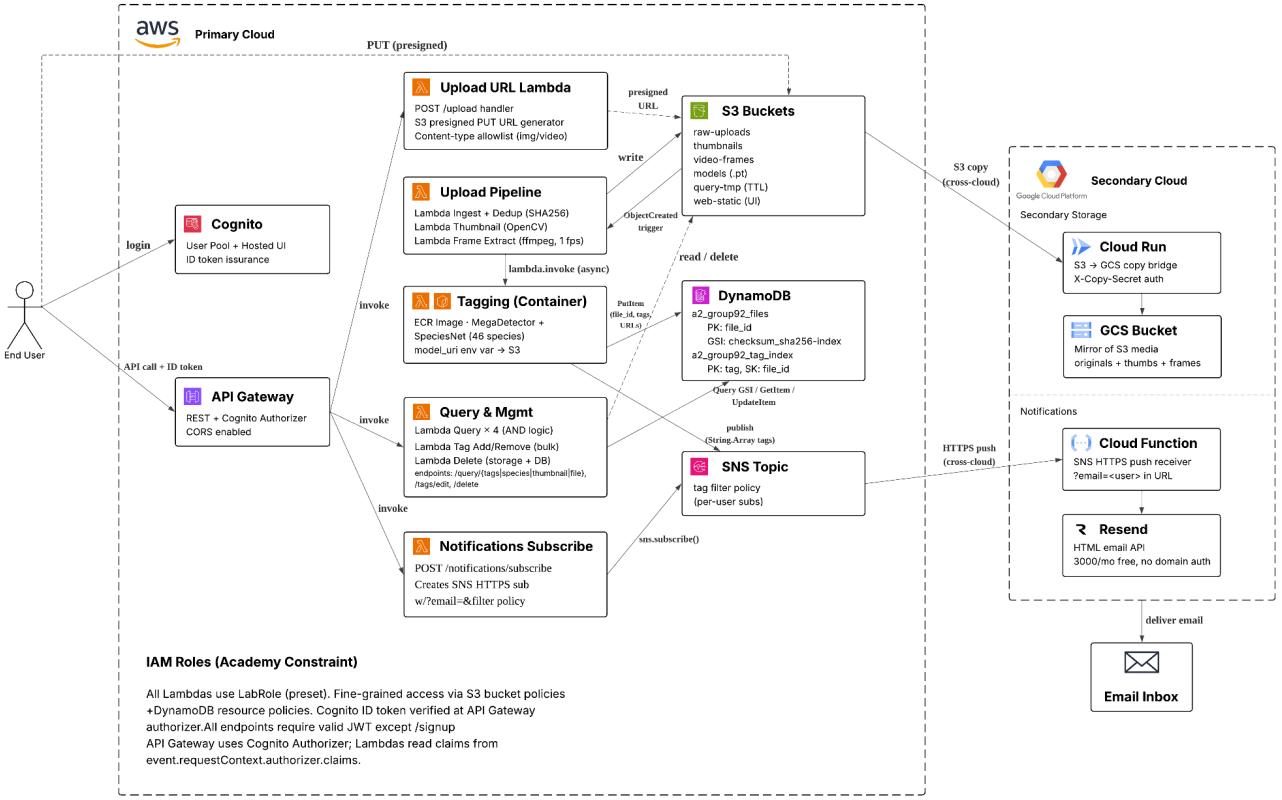


Figure 1: Multi-Cloud architecture diagram of the platform

II. Roles of Team Members

Name & ID	Contribution Ratio	Project Elements
Yuanhe Wu 36302414	28%	• AWS Cognito User Pool + Hosted UI with Aussie-themed branding (Auth Code + PKCE); ID Token verification at API Gateway • API Gateway: 8 cognito-authorized REST routes, CORS preflight + 4xx/5xx Gateway Responses CORS, binary media types • `/upload` Lambda: SHA-256 deduplication via DynamoDB GSI + presigned S3 PUT URL • Multi-cloud notifications: SNS Topic (String.Array OR), subscribe Lambda (per-user HTTPS with URL-encoded email, tag normalisation), GCP Cloud Function (Python 3.12 Gen 2) for SNS auto-confirm + tag-event/welcome emails via Resend • Cross-team integration: redeploying stale Lambda zips, patching CORS responses, restoring OAuth config • Multi-cloud architecture diagram (LucidChart), themed UI styling, `shared_cognito_auth.md`
Guanghui Liu 34217398	24%	• Implemented the P2 file handling pipeline for the multi-cloud serverless wildlife platform, including AWS S3 upload, Lambda-triggered media processing, and DynamoDB file metadata storage. • Built SHA-256 checksum deduplication, image thumbnail generation, and ffmpeg-based video frame extraction for uploaded media files. • Integrated GCP Cloud Storage as the secondary cloud provider by mirroring S3 media objects and recording secondary storage URLs/status fields in DynamoDB. • Implemented and tested the delete workflow to remove original files, thumbnails, video frames, database records, tag index entries, and mirrored GCP objects.

Junzhe Zhang 34682457	25%	• Built and deployed the ML tagging Lambda container through AWS ECR. • Implemented image species detection and detect-only mode for query-by-file. • Integrated tagging results with DynamoDB files and tag_index tables. • Fixed video tagging by collecting extracted video frames, running frame-level species detection, aggregating species counts back to the original video records, and making videos searchable by detected species tags. • Updated and tested query responses for AWS presigned URLs, GCP secondary storage fields, video frame URLs, and frame counts. • Tested and debugged /query/tags, /query/species, /query/file, tag editing, SNS notification logic, multi-cloud response fields, and video species query results.
Zuyao Wang 34590706	23%	• Built the frontend dashboard for upload, media results, queries, tag management, deletion, and notifications. • Integrated AWS Cognito Hosted UI login/logout, callback handling, token storage, and authenticated API requests. • Connected frontend functions to cloud API Gateway endpoints, including presigned S3 upload, Query by Tags, Query by File, and thumbnail lookup. • Improved Media Results with thumbnail/video previews, tag chips, collapsed technical details, and copy/open actions. • Added toast messages, friendly outputs, error handling, and tested/debugged integration using Chrome DevTools and GitLab merge requests.

Table 1: Roles of team members of Group 92

III. User Guide

This guide explains how to use **Aussie EcoLens: A Multi-Cloud Serverless Wildlife Observation Platform** to upload, search, manage, and subscribe to wildlife media records.

To use this application, open `frontend/index.html` after cloning the repo.

1. Accessing the Application

Open the web dashboard and click **Login with Cognito**. After successful login, the dashboard shows the authenticated user status, and API requests automatically include the Cognito token. If the session expires, users should log out and log in again.

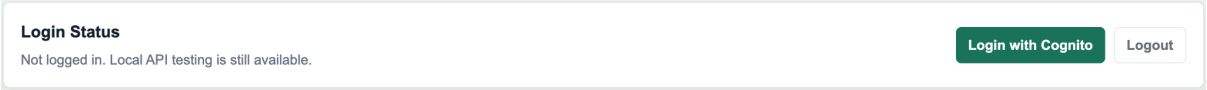


Figure 2: Login status panel

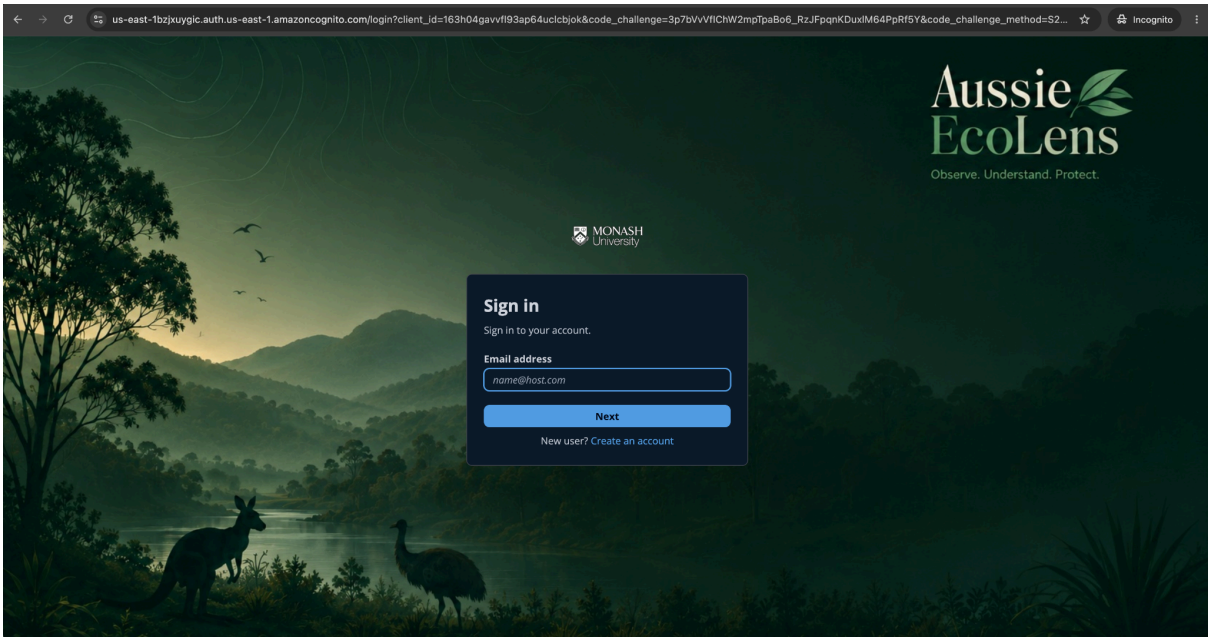


Figure 3: Login page

2. Uploading Media

Go to **Upload Media to Library**, choose an image or video, and click **Upload**. The system requests a presigned S3 upload URL from the cloud API, uploads the file to storage, and then triggers backend processing. Uploaded images generate thumbnails, videos generate frame previews, and detected species tags are stored in the database.

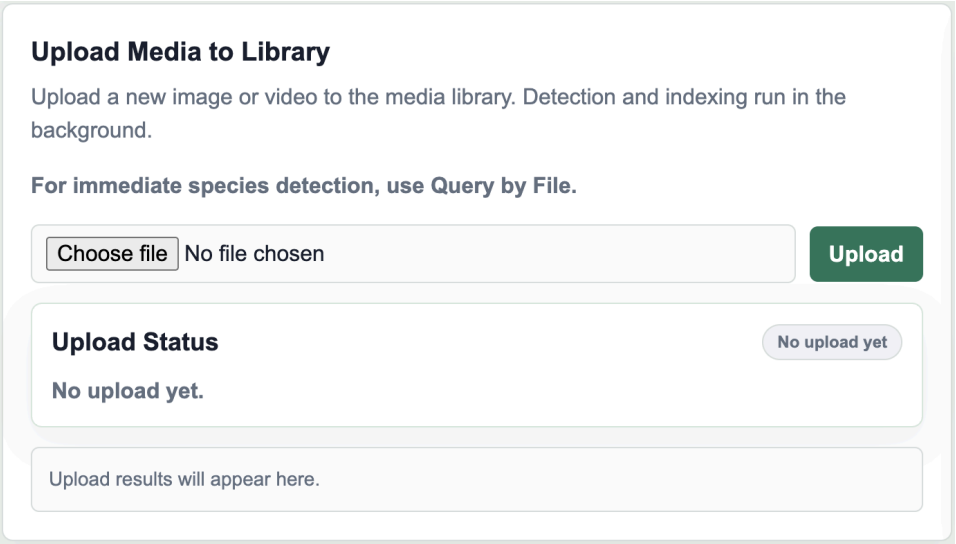


Figure 4: Media uploading panel

If the same file is uploaded again, checksum-based deduplication detects it and shows a **Duplicate** message.

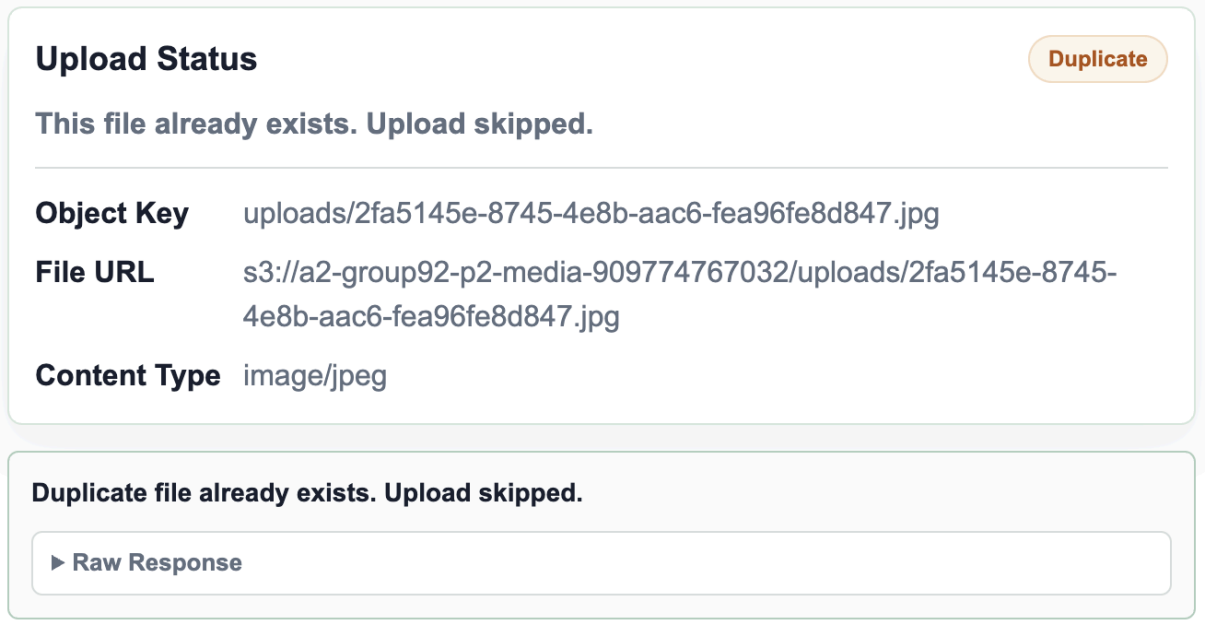


Figure 5: Upload status of duplicate case

3. Searching by Species or Tags

Use **Query by Species/Tags** to search stored media by detected or manually added tags. The input supports both species-only and tag-count queries, such as `bos_taurus`, `bos_taurus=1`, or `wombat=2, magpie=1`. Multi-tag queries use logical **AND**, so all requested tag conditions must be satisfied. This search is case-insensitive.

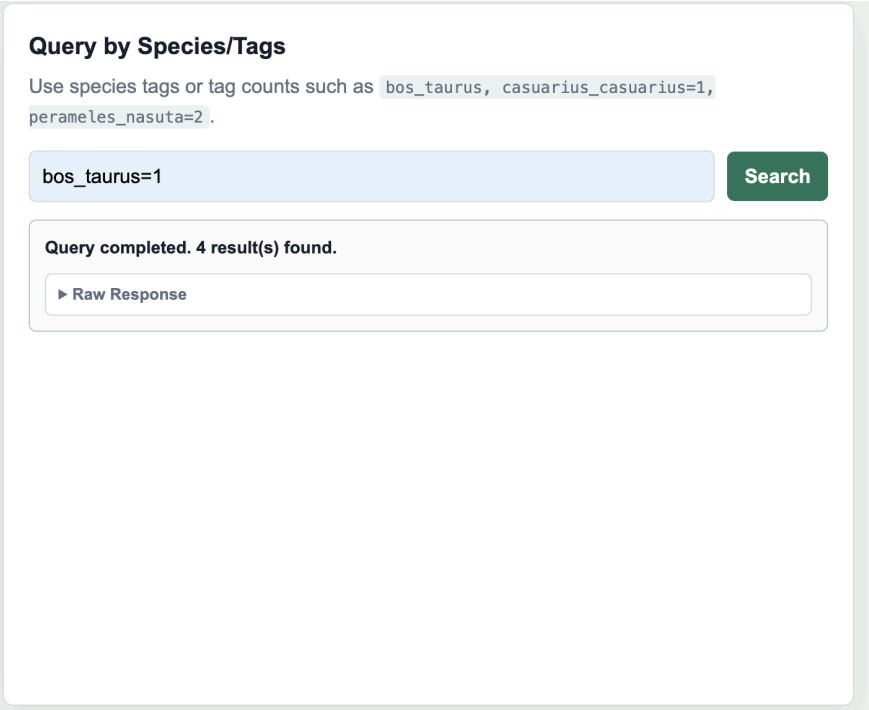


Figure 6: Query by Species/Tags panel

4. Media Results

The **Media Results** section displays returned records as media cards. Images use thumbnail previews to save bandwidth, while videos can show extracted frame previews. Detected species and system tags are displayed as readable chips, such as **`Bos taurus × 1`** or **`Video file × 1`**. Long technical values, including file URLs, thumbnail URLs, frame URLs, object keys, and raw JSON, are hidden under **Technical Details** and can be expanded when needed.

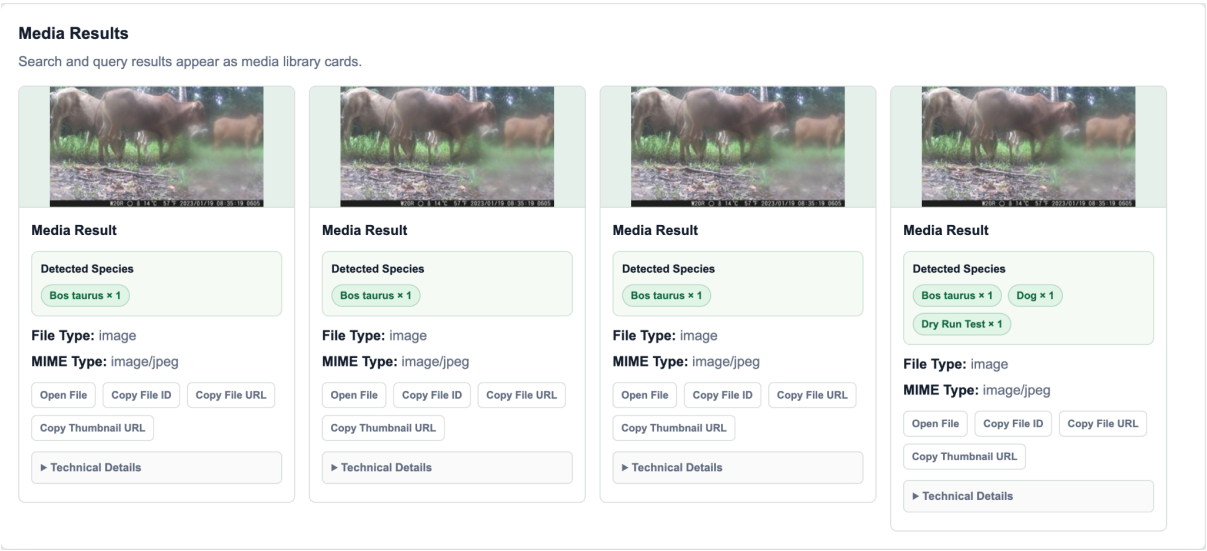


Figure 7: Media Results panel

5. Searching by Uploaded Query File

Use **Query by File** when the user does not know the tag name. Select a temporary query image and click **Search by File**. The system detects species in the query image and searches the database for matching media. The query file is not permanently added to storage or the database. For reliable testing, use a small image, as very large files may exceed API request size limits.

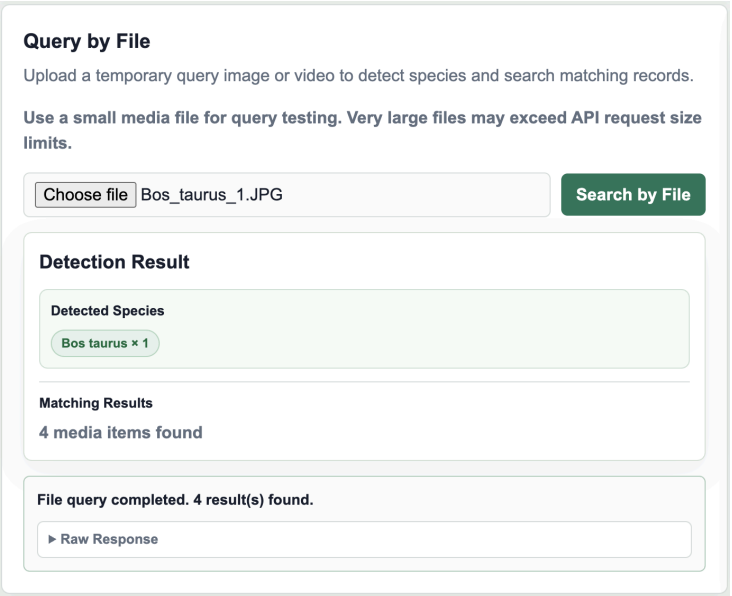


Figure 8: Query by File panel

6. Searching by Thumbnail URL

Use **Query by Thumbnail URL** to find the original full-size media file from a thumbnail. Paste a full thumbnail URL or **`/thumbnails/...`** path and click **Find Original File**. The frontend normalises long presigned URLs before sending the request. Clicking "Open Original File" will open a new tab in the browser to display a larger image.

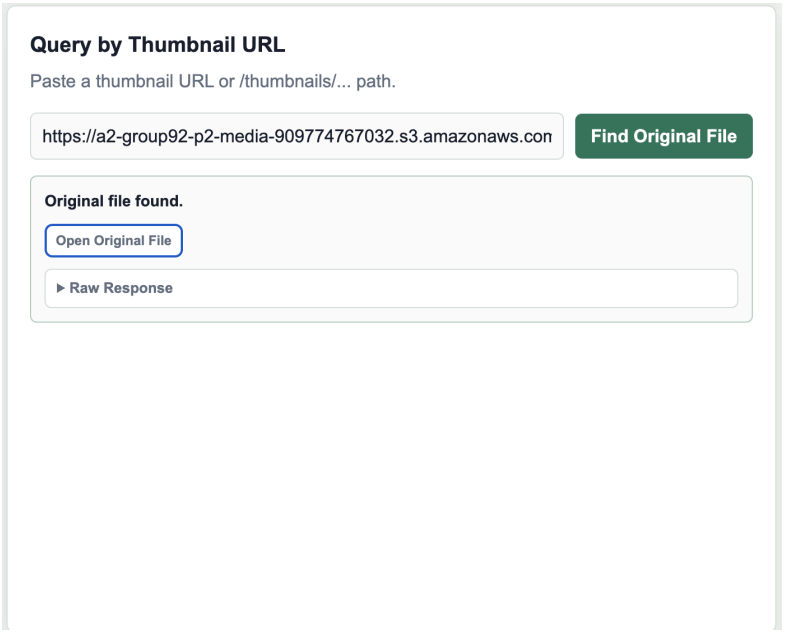


Figure 9: Query by Thumbnail URL panel

7. Tag Management and Deletion

Use **Tag Management** to add, update, or remove tags from uploaded files by entering the file URL, tag name, count, and operation. After adding a tag, search it in **Query by Tags** to verify the update.

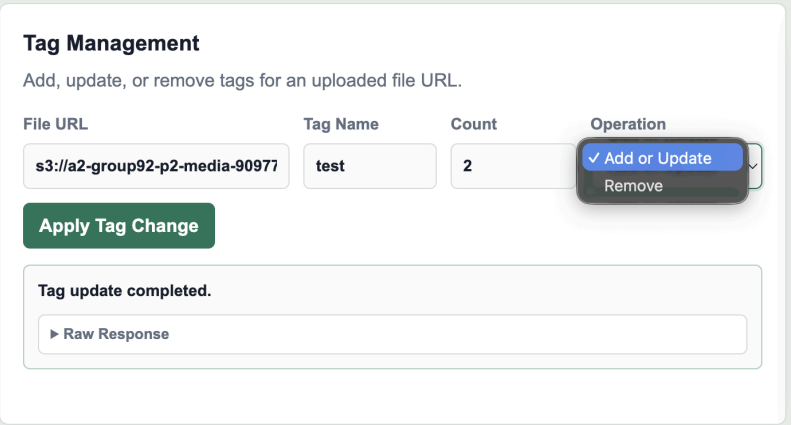


Figure 10: Tag Management panel

Use **Delete File** to remove test media records, related thumbnails or video frames, and database entries. During the demonstration, only delete the test files.

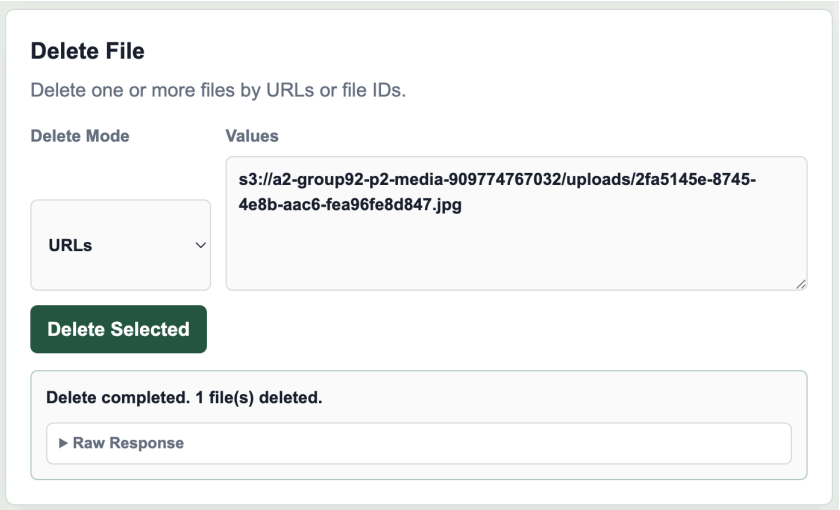


Figure 11: Delete File panel

8. Notifications and Troubleshooting

Use **Notification Subscription** to subscribe to updates for selected wildlife tags. If a request fails, expand **Technical Details** or **Raw Response** to inspect the API response. If a new upload does not appear immediately in search results, wait for the asynchronous processing pipeline to finish and try again.

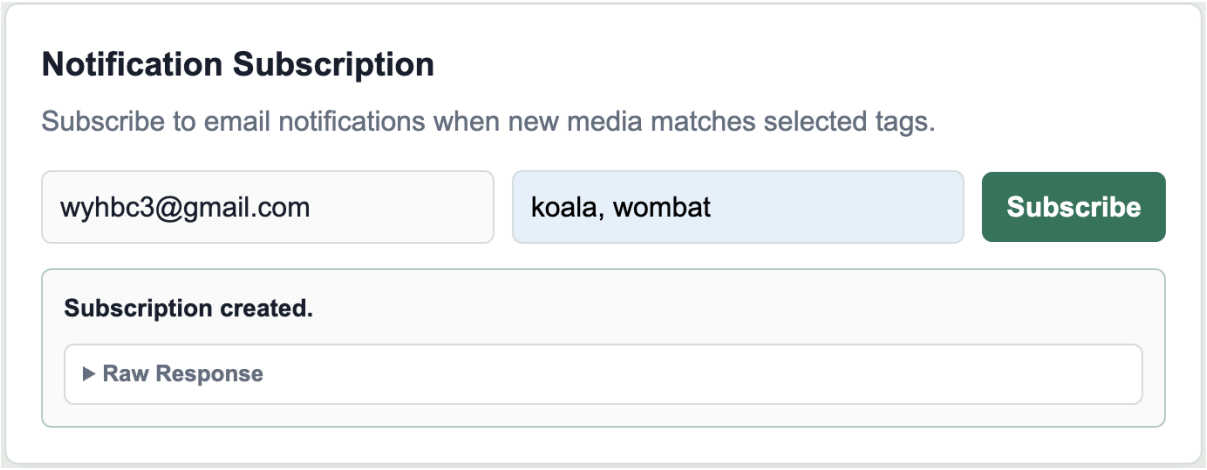


Figure 12: Notification Subscription panel

A confirmation email will be received when subscribing to a new tag.

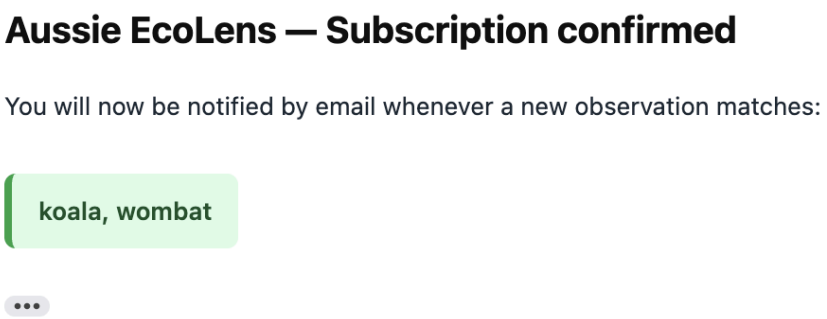


Figure 13: Subscription confirmation email

A notification email will be received when a new image is uploaded to the subscribed tag.

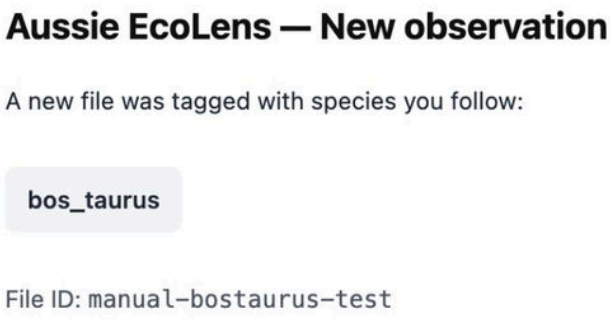


Figure 14: New observation email

IV. Link to Source Code (GitLab)

https://git.infotech.monash.edu/fit5225a2-group92/assignment2

AI Acknowledgment

This project used generative AI tools to support implementation, debugging, deployment, and documentation. All AI-generated outputs were reviewed, tested, and adapted by the team before being integrated into the deliverables, and all final architectural, design, and code decisions remain the responsibility of the four authors named on the cover page.
Tools used:

- Anthropic Claude and OpenAI Codex: Used to assist in the development of their respective modules, such as assisting in writing some AWS Lambda and GCP cloud function code, S3 data import pipeline + secondary GCS storage bridging, MegaDetector + SpeciesNet tag container, and front-end UI. In addition, these GenAI tools were used for debugging multi-cloud integrations, troubleshooting AWS and GCP consoles, and polishing architecture diagrams and reports.
- OpenAI ChatGPT: Used only to generate two sample background images: the Cognito Hosted UI login page background and the dashboard sidebar background.

References

Amazon Web Services, "Amazon Cognito User Pools — Hosted UI," https://docs.aws.amazon.com/cognito/latest/developerguide/cognito-user-pools-app-integration.html

Amazon Web Services, "Use Amazon Cognito User Pools as an authorizer for Amazon API Gateway," https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-integrate-with-cognito.html

Amazon Web Services, "Amazon SNS message filtering (String.Array)," https://docs.aws.amazon.com/sns/latest/dg/sns-message-filtering.html

Amazon Web Services, "Verifying the signatures of Amazon SNS messages," https://docs.aws.amazon.com/sns/latest/dg/sns-verify-signature-of-message.html

Amazon Web Services, "Generating a presigned URL to upload an object to S3," https://docs.aws.amazon.com/AmazonS3/latest/userguide/PresignedUrlUploadObject.html

Google Cloud, "Cloud Functions (2nd gen) HTTP triggers," https://cloud.google.com/functions/docs/calling/http

Google Cloud, "Cloud Run — service-to-service authentication," https://cloud.google.com/run/docs/authenticating/service-to-service

IETF RFC 7636, "Proof Key for Code Exchange by OAuth Public Clients," https://datatracker.ietf.org/doc/html/rfc7636

Beery, S., Morris, D., & Yang, S., "MegaDetector," https://github.com/agentmorris/MegaDetector
Google Research, "SpeciesNet," https://github.com/google/cameratrapai
Resend, "Resend API documentation," https://resend.com/docs
FFmpeg developers, "FFmpeg documentation," https://ffmpeg.org/documentation.html